Software Requirements Specification

- **Version 1.0**

Political Candidate Information Platform

- Stefan — Product Manager / Frontend Developer
- Avery — QA / UX
- Dimitri — Backend Developer

CSC 2920 — Software Engineering

UNIVERSITY of NORTH CAROLINA
PEMBROKE

Table of Contents

Software Requirements Specification 1
Table of Contents 2
List of Figures 3
1.0 Introduction 3
1.1 Purpose 3
1.2 Scope of Project 3
1.3 Glossary 3
1.4 References 4
1.5 Overview of Document 4
2.0 Overall Description 5
2.1 System Environment 5
2.2 Functional Requirements Specification 7
2.2.1 Voter Use Case 7
2.2.2 Voter Use Case 8
2.2.3 Admin Use Case 8
2.2.4 System Use Case 11
2.3 User Characteristics 12
2.4 Non-Functional Requirements 12
3.0 Requirements Specification 12
3.1 External Interface Requirements 12
3.1.1 DTOs and API Contracts 13
3.2 Functional Requirements 15
3.2.1 View Candidate Profile 15
3.2.2 Search Candidates 16
3.2.3 Sync Congressional Data 16
3.2.4 Retrieve Sponsored Bills 16
3.3 Detailed Non-Functional Requirements 17
3.3.1 Logical Structure of the Data 17
3.3.2 Security 19
3.3.3 Postman Collection and Test Cases 19
3.3.4 JUnit Testing 19
3.3.5 Error Handling and Logging 20
3.3.6 Data Validation Rules 20
4.0 Other Requirements 20

4.1 References 20
4.2 Complying with Election Laws 20
4.3 No Advertisements for Parties 21

List of Figures

- Figure 1 — System Environment
- Figure 2 — Use-Case Diagram
- Figure 3 — Data Synchronization Process
- Figure 4 — Logical Structure of Candidate Data

1.0 Introduction

1.1 Purpose

The purpose of this document is to present a detailed description of the Political Candidate Information Platform. It explains the system's purpose, features, interfaces, constraints, and behavior under various conditions. This document is intended for stakeholders, developers, testers, and instructors evaluating the system.

1.2 Scope of Project

This software system provides voters with structured, unbiased information about political candidates. It consolidates data from the Congress.gov API and presents it through a modern, responsive web interface.

The system enables users to:

- Browse candidate profiles
- View background, experience, priorities, strengths, and weaknesses
- Search and filter candidates
- Explore sponsored and cosponsored legislation
- Access structured, reliable political information

The backend uses Spring Boot and MySQL, while the frontend uses React.js.

1.3 Glossary

Term	Definition
Candidate Profile	A structured record containing background, experience, priorities, and legislative activity.
Congress.gov API	The external data source providing congressional member and bill information.
Voter	A general user accessing the platform to view candidate information.
Admin	A system operator responsible for triggering data synchronization and monitoring system health.
DTO	Data Transfer Object used to move structured data between layers.
REST API	A web service architecture used by the backend to communicate with the frontend.
Use Case	A description of system behavior from the perspective of an actor.

1.4 References

- IEEE Std 830-1998 — Recommended Practice for Software Requirements Specifications
- Use-Case 2.0: The Guide to Succeeding with Use Cases
- Congress.gov API Documentation

1.5 Overview of Document

- Section 2.0 provides an overview of the system's environment and informal requirements.

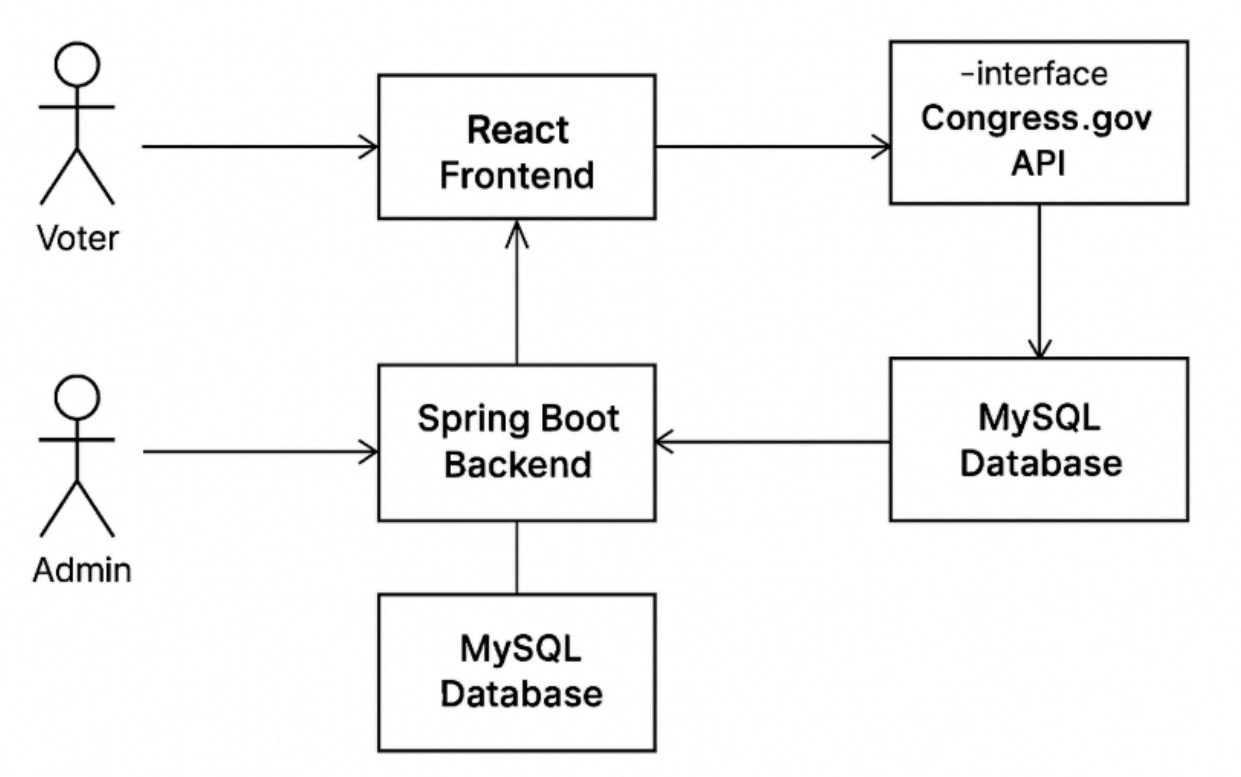
- Section 3.0 provides detailed functional and non-functional requirements for developers.
- Section 4.0 provides other requirements for developers.

2.0 Overall Description

2.1 System Environment

Figure 1 — System Environment

- This software system provides voters with structured, unbiased information about political candidates." "The backend uses Spring Boot and MySQL, while the frontend uses React.js.
- The system integrates data from Congress.gov to populate candidate profiles and legislative records. In addition to member and bill metadata, the backend ingests bill summaries (HTML/CDATA) from the Congress.gov Bill Summaries API and stores them in the application database for display and auditing.



Actors:

- **Voter** — Accesses the platform through a web browser

- **Admin** — Manages data synchronization
- **Congress.gov API** — External data provider
- **Database (MySQL)** — Stores structured candidate information
- **Backend (Spring Boot)** — Processes requests and manages data
- **Frontend (React.js)** — Displays UI to users

The system is divided into two major components:

1. **Public Candidate Portal** — Used by voters
2. **Data Management Backend** — Used by admins and automated processes
3. **New data source**: *Congress.gov Bill Summaries API* **(`/bill/{congress}/{billType}/{billNumber}/summaries`)**. The backend must support XML and JSON responses and preserve the original HTML/CDATA summary text for display and auditing.
4. **Parsing note**: XML responses may include CDATA-wrapped HTML; the ingestion pipeline must extract and sanitize the HTML before storing.

Goals :

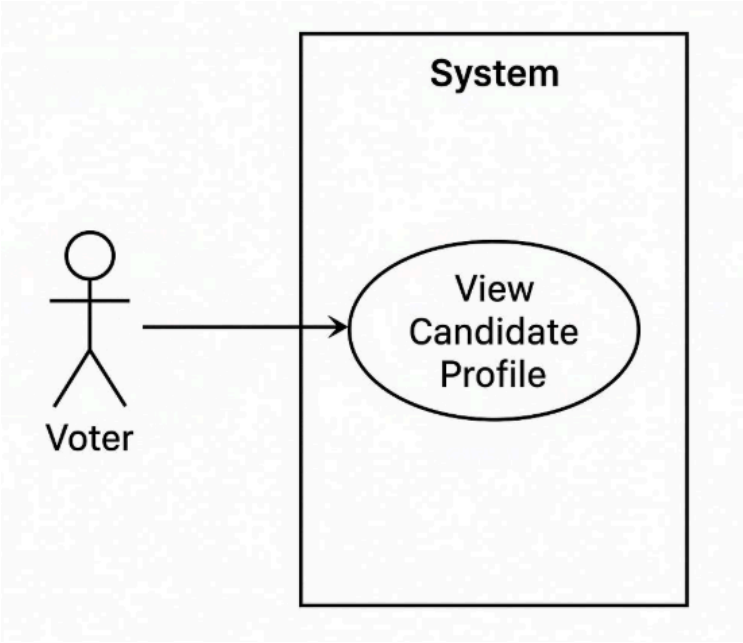
- Help People Conduct Research — This will help voters conduct research in a short amount of time, as they will not need to spend extra time searching for reliable information. It allows voters to quickly access the facts they need and make informed Decisions.
- Reduce the Spread of Misinformation — This will encourage voters to seek accurate information about the candidates they are voting for and better understand which political party aligns with their views. By promoting fact-based research instead of relying on hearsay, it will help prevent the rapid spread of misinformation.
- Encourage Youth Voting — Allowing and encouraging youth participation in voting will motivate young people to engage in political discussions with peers and family members. This will help them better understand current events and the real world, while also strengthening their academic research and critical thinking skills through active information gathering

2.2 Functional Requirements Specification

2.2.1 Voter Use Case

Use Case: View Candidate Profile

Figure 2 — Diagram:



Brief Description:

The voter views a structured candidate profile containing background, experience, priorities, and legislative activity.

- **Initial Step-By-Step Description:**

Before this use case can be initiated, the voter has accessed the platform.

1. The voter selects a candidate from the list.
2. The system retrieves candidate data from the database.
3. The system displays the candidate's profile.
4. The voter may navigate to related bills or issues.

2.2.2 Voter Use Case

Use Case: Search Candidates

- **Brief Description:**

The voter searches for candidates using name, state, party, or issue filters.

Initial Step-By-Step Description:

1. The voter enters a search term or selects filters.
2. The system processes the query.
3. The system displays matching candidates.
4. The voter selects a candidate to view details.

2.2.3 Admin Use Case

Use Case: Sync Congressional Data

- **Brief Description:**

  Purpose Define the automated and admin-triggered process that synchronizes member, legislation, and bill summary data from Congress.gov into the application database, ensuring provenance, idempotency, and safe rendering of HTML summaries.

  1. For each `Legislation` record, call the Bill Summaries API using `(congress, type, number)`.
  2. Validate the response and extract `text` (HTML), `versionCode`, `actionDate`, `actionDesc`, `updateDate` when present.
  3. Link or insert a `BillSummary` row referencing the parent `Legislation` record.

The admin triggers a synchronization process to update candidate data from Congress.gov..

Preconditions

- Backend has network access to Congress.gov and valid API credentials.
- Legislation records exist or are created during member/bill ingestion.
- Rate-limiting and retry configuration are set in environment variables.

Basic Flow (ordered, testable steps)

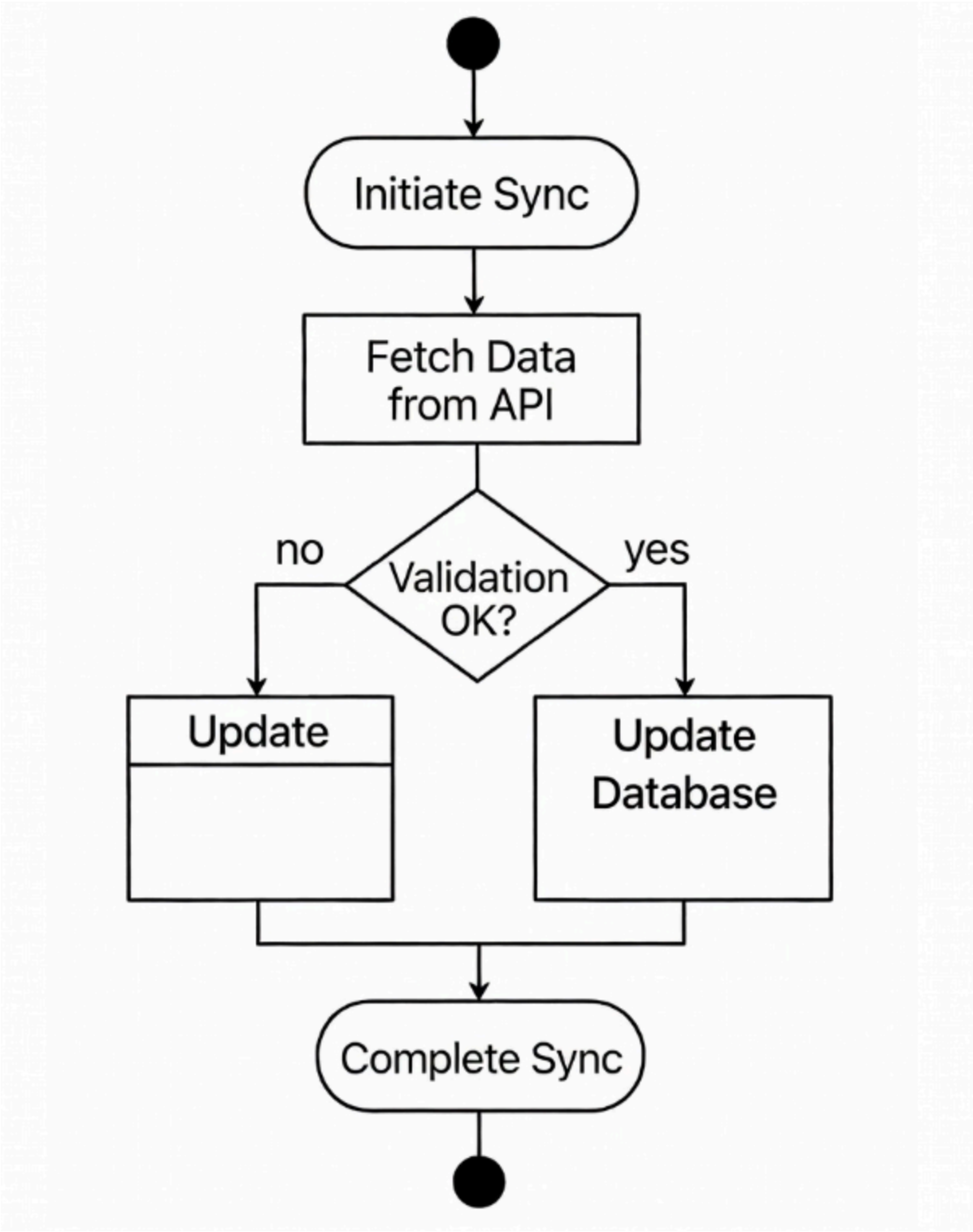
1. **Discover work items:** determine legislation rows to sync (new, updated, or flagged for re-sync).

2. **Throttle:** apply per-host concurrency limits and rate-limiting to respect Congress.gov policies.
3. **Lookup parent:** for each work item, locate Legislation using the composite key (congress, type, number). If not found, optionally create a placeholder legislation record or log and skip, per configuration.
4. **Fetch summaries:** call **`/bill/{congress}/{billType}/{billNumber}/summaries`** (accept JSON).
5. **Validate response:** confirm returned **(congress, billType, billNumber)** matches the request; if mismatch, log and abort that item.

Acceptance criteria (must be testable)

- **AC-1:** For a given (congress,type,number), a successful sync results in at least one bill_summary row with text populated and legislation_id set.
- **AC-2:** Stored text renders in the frontend without script execution (sanitizer test pass).
- **AC-3:** Re-running the same sync is idempotent (no duplicate summaries for same versionCode and fetched_at).
- **AC-4:** Sync job records provenance (source, fetched_at, raw_response_pointer) for each ingested summary.
- **AC-5:** Sync metrics and logs are emitted and visible in monitoring dashboards within 5 minutes of job completion.

Figure 3 — Data Synchronization Process



Initial Step-By-Step Description:

1. Admin selects “Sync Data.”
2. The system sends a request to Congress.gov..

3. The system retrieves member data.
4. The system updates the database.
5. The system logs the sync event.

2.2.4 System Use Case

Use Case: Retrieve Sponsored Bills

- **Brief Description:**

Purpose

Define how the system retrieves, merges, and displays sponsored and cosponsored bills for a candidate, and how bill summaries are included in the results.

Trigger

- Voter selects **View Sponsored Bills** on a candidate profile.

Preconditions

- Candidate exists in the database and has associated Legislation records (sponsored and/or cosponsored).
- BillSummary ingestion has run at least once for the relevant legislation or is available on demand.

Basic flow (ordered, testable steps)

1. **Request**: frontend calls **GET api/members/S001209/legislation**
2. **Lookup candidate**: backend resolves bioguide_id → candidate record and retrieves associated Legislation rows (apply filters, pagination, and sorting).
3. **Fetch summaries**: for each legislation row returned, include the latest BillSummary (sanitized text).

Acceptance criteria (must be testable)

- **AC-1:** Endpoint returns combined sponsored and cosponsored bills for a candidate with deduplication by **`(congress,type,number)`.**
- **AC-2:** Each bill includes `summary` when available; summary.text is sanitized and safe to render.
- **AC-3:** Pagination and sorting behave as specified.
- **AC-4:** On-demand summary fetches queue background ingestion and does not block the primary response.

2.3 User Characteristics

- **Voter:** Internet-literate, able to use search and navigation tools.
- **Admin:** Familiar with basic system operations and monitoring tools.
- **Developer:** Experienced with Java, Spring Boot, React, and SQL.

2.4 Non-Functional Requirements

- The system must run on modern browsers.
- The backend must respond within 2 seconds under normal load.
- The system must use HTTPS for all communication.
- The UI must be responsive and accessible.

3.0 Requirements Specification

3.1 External Interface Requirements

1. Defines the external and internal HTTP interfaces used to ingest bill summaries from Congress.gov and to serve bill and summary data to the frontend.

The system interfaces with:

- **Congress.gov API** — Retrieves congressional member and bill data
    1. **Endpoint pattern**: https://api.congress.gov/v3/bill/{congress}/{billType}/{billNumber}/summaries
    2. **Authentication**: use API key or OAuth as required by Congress.gov; credentials stored in secrets manager.
    3. **Validation**: verify returned (congress, billType, billNumber) matches the request.
- **MySQL Database** — Stores structured candidate information
- **React Frontend** — Displays UI and sends requests to backend

Acceptance criteria (API-level)

- Internal GET /api/bills/{congress}/{type}/{number}/summaries returns legislation and zero-or-more summaries with sanitized text.

3.1.1 DTOs and API Contracts

- **Overview** — DTOs represent the JSON payloads exchanged between frontend and backend. Each DTO must include field names, types, nullability, and example values. DTOs are immutable where possible and validated server-side using bean validation annotations.
- **MemberDto**

```
package com.csc2920.group_project.dto;

import com.fasterxml.jackson.annotation.JsonIgnoreProperties;
import lombok.Data;
import lombok.NoArgsConstructor;

import java.util.List;

@Data
@NoArgsConstructor
@JsonIgnoreProperties(ignoreUnknown = true)
public class MemberDto {

    private String bioguideId;
    private String name;
    private String state;
    private Integer district;
    private String partyName;
    private Terms terms;
    private Depiction depiction;
    private String updateDate;

    @Data
    @NoArgsConstructor
    @JsonIgnoreProperties(ignoreUnknown = true)
    public static class Terms {
        private List<Term> item;
    }

    @Data
    @NoArgsConstructor
    @JsonIgnoreProperties(ignoreUnknown = true)
    public static class Term {
        private Integer startYear;
        private String chamber;
    }
```

```
    @Data
    @NoArgsConstructor
    @JsonIgnoreProperties(ignoreUnknown = true)
    public static class Depiction {
        private String attribution;
        private String imageUrl;
    }
}
```

- **LegislationDto**

```
package com.csc2920.group_project.dto;

import com.fasterxml.jackson.annotation.JsonIgnoreProperties;
import lombok.Data;
import lombok.NoArgsConstructor;

@Data
@NoArgsConstructor
@JsonIgnoreProperties(ignoreUnknown = true)
public class LegislationDto {

    private Integer congress;
    private String introducedDate;
    private LatestAction latestAction;
    private String number;
    private PolicyArea policyArea;
    private String title;
    private String type;
    private String url;
    private String source; // e.g., "sponsored" | "cosponsored"

    @Data
    @NoArgsConstructor
    @JsonIgnoreProperties(ignoreUnknown = true)
    public static class LatestAction {
        private String actionDate;
        private String text;
    }

    @Data
    @NoArgsConstructor
    @JsonIgnoreProperties(ignoreUnknown = true)
    public static class PolicyArea {
        private String name;
```

```
    }
}
```

- **BillSummaryDto**

```
package com.csc2920.group_project.dto;

import com.fasterxml.jackson.annotation.JsonIgnoreProperties;
import lombok.Data;
import lombok.NoArgsConstructor;

@Data
@NoArgsConstructor
@JsonIgnoreProperties(ignoreUnknown = true)
public class BillSummaryDto {

    private String number;    // billNumber from request, e.g., "3076"
    private String type;      // billType from request, e.g., "s" or "HR"
    private Integer congress; // congress from request, e.g., 117
    private String text;      // sanitized HTML extracted from CDATA in <text>
}
```

3.2 Functional Requirements

- This section specifies the system behaviors required to support candidate profiles, search, data synchronization with Congress.gov, and storage and retrieval of bill summaries.

3.2.1 View Candidate Profile

- **Trigger:** Voter selects a candidate

  The system shall return a candidate profile for a valid bioguide_id containing member fields (bioguide_id, name, party_name, state, district, update_date), associated Terms, Depiction, and a paginated list of Legislation records.

- **Precondition:** Candidate exists in database
- **Store and serve bill summaries**: The system shall store `congress`, `billType`, `billNumber`, and `text` (raw HTML) for each summary and serve it via the internal API tied to `Legislation`.

Basic Path:

1. System retrieves candidate data

2. System displays profile

Postcondition: Candidate profile is shown

3.2.2 Search Candidates

- **Trigger:** Voter enters search criteria
    1. The system shall support search by name, and bioguide_id with server-side pagination and sorting.
    2. Search results shall include a candidate summary card with a link to the full profile and sponsored/cosponsored bills.

Basic Path:

1. System processes search
2. System returns matching candidates

3.2.3 Sync Congressional Data

- **Trigger:** Admin initiates sync
    1. The system shall fetch member and legislation metadata from Congress.gov and upsert into member and legislation tables, enforcing UNIQUE(congress, type, number) on legislation.
    2. For each Legislation record, the system shall call the Bill Summaries API using (congress, type, number) and process the response per the ingestion workflow in Section 2.2.3.
    3.

Basic Path:

1. System fetches data
2. System updates database

3.2.4 Retrieve Sponsored Bills

- **Trigger:** Voter selects “View Bills”
    1. The system shall return combined sponsored and cosponsored bills for a candidate, deduplicated by (congress, type, number), annotated with role (sponsor | cosponsor).
    2. Each returned bill shall include a summary when available.

Basic Path:

1. System retrieves sponsored and cosponsored bills
2. System merges results

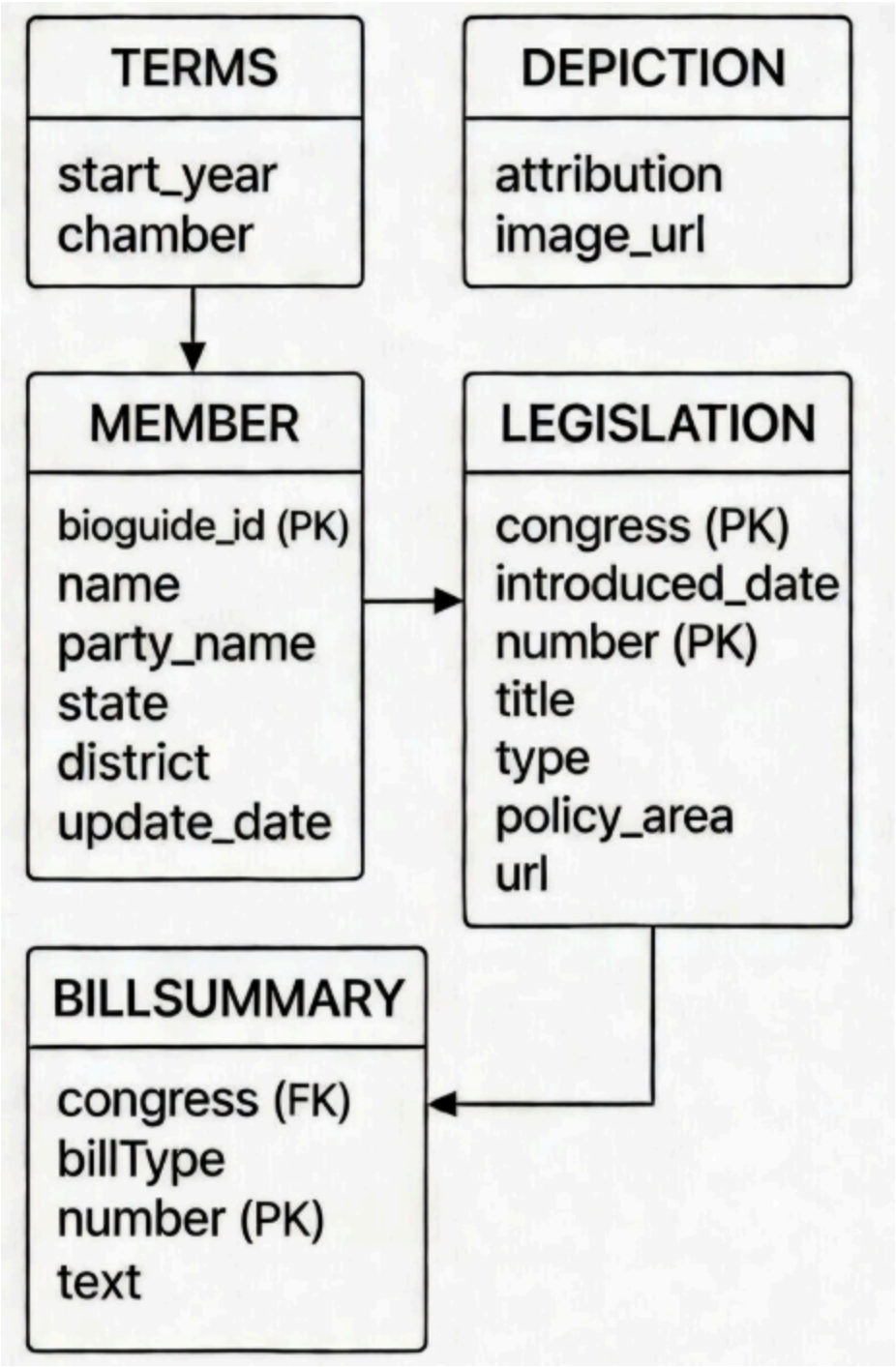
3. System displays list

3.3 Detailed Non-Functional Requirements

Scope This section defines non-functional constraints and quality attributes that apply to the system components involved in member, legislation, and bill summary ingestion, storage, and delivery.

3.3.1 Logical Structure of the Data

Figure 4 — Logical Structure of Candidate Data



Tables include:

- Candidate
- Legislation(Sponsored / Cosponsored)
- Bill Summary
- Issues
- Sync Logs

3.3.2 Security

- All endpoints must require HTTPS
- Input validation must prevent injection
- Admin actions must require authentication

3.3.3 Postman Collection and Test Cases

Purpose

Provide a repeatable, shareable Postman collection for API contract testing, exploratory testing, and automated checks in CI.

Postman Collection and Test Cases

Collection structure — group requests by resource: **Members**, **Legislation**, **BillSummaries**, **Admin**. Include environment variables for **baseUrl**, **apiKey**, and **authToken**.

Essential requests

- **GET /api/members/{bioguide_id}** — validate **200**, response schema, presence of terms and legislation array.
- **GET /api/members/{bioguide_id}/legislation** — validate pagination, summaryAvailable flags.
- **GET /api/bills/{congress}/{type}/{number}/summaries** — validate returned summaries array and sanitized text.
- **POST /admin/sync** (admin only) — validate job accepted (**202**) and return job id.

Test scripts and assertions

- **Status checks** — assert expected HTTP status codes (200, 202, 404 where applicable).
- **Schema validation** — validate responses against stored JSON schemas.

3.3.4 JUnit Testing

- Unit tests validate parsing, sanitization, DTO mapping, and service-layer logic.
- Integration tests validate repository behavior, unique constraints, and ingestion idempotency.
- Frameworks: JUnit 5, Mockito, Spring Boot Test.
- Required test categories:
    1. Parser tests (HTML preservation)
    2. Sanitizer tests (remove <script>, on* attributes, unsafe URLs)
    3. Idempotency tests for BillSummary ingestion
    4. DTO mapping tests
    5. Controller tests using @WebMvcTest

6. Integration tests using H2 for DB upsert validation

3.3.5 Error Handling and Logging

- All ingestion failures (network, schema mismatch, invalid bill data) are logged.
- Validation errors return structured JSON with errorCode, message, and timestamp.
- All admin-triggered sync operations are logged.

3.3.6 Data Validation Rules

- All external API responses are validated for required fields (congress, billType, billNumber).
- BillSummary text must pass HTML sanitization before storage.
- Member and Legislation records must satisfy unique constraints.
- API requests from frontend validate:
    1. bioguide_id format
    2. pagination parameters
    3. bill identifiers (congress, type, number)

4.0 Other Requirements

4.1 References

The system includes references to:

- Congress.gov API — Retrieves congressional member profiles, legislation data, images, and related metadata

4.2 Complying with Election Laws

- **Legal Compliance** — The application will comply with all applicable federal and state election laws
- **No Exploitation** — Users will not be able to access, manipulate, or exploit the system in any way that would violate election regulations
- **Safeguards** — The platform will implement measures to prevent misuse of information related to voting, elections, or candidate promotion
- **Privacy** — do not store or expose personal data beyond what is required for candidate profiles; follow institutional privacy policies and applicable laws.
- **Content neutrality** — the platform shall not display paid political advertising or promotional content; all candidate content must be presented neutrally.

4.3 No Advertisements for Parties

- **No Political Advertising** — The application will not display advertisements, sponsored content, or any form of promotional material for political parties, candidates, or affiliated organizations
- **Neutrality** — Candidates and affiliated organizations will not be permitted to influence voters through advertisements or promotional messaging within the application
- **Focus** — The platform will remain strictly neutral and focused solely on providing factual, research-based information